

Abstração de Dados e Estruturas Lineares em C

Listas, Pilhas, Filas e Árvores

Prof. Me. Luis Vinicius Costa Silva

NF2 – MAPEAMENTO E DESENVOLVIMENTO DE BANCO DE DADOS
Centro Universitário UniAtenas
Análise e Desenvolvimento de Sistemas – EaD

16 de março de 2026



Objetivos da Aula

- ▶ Compreender abstração de dados e Tipos Abstratos de Dados (TAD)
- ▶ Dominar conceitos, operações e complexidades de:
 - ▶ Listas (sequencial e encadeada)
 - ▶ Pilhas (LIFO)
 - ▶ Filas (FIFO)
 - ▶ Árvores (TAD para hierarquias)
- ▶ Identificar quando usar cada estrutura
- ▶ Representar graficamente o estado das estruturas
- ▶ Escrever pseudocódigo e código em C das operações principais

Problema Motivador

Problema: Gerenciar um sistema de tarefas, onde cada tarefa pode ser adicionada, removida ou consultada, e algumas tarefas têm prioridade.

Perguntas:

- ▶ Um vetor simples atende bem?
- ▶ Como organizar tarefas em ordem de chegada ou prioridade?
- ▶ Que estrutura usar para histórico de ações (undo)?

Objetivo: Mostrar que TADs e estruturas lineares permitem modelar soluções eficientes.

Abstração de Dados

- ▶ Esconder detalhes de implementação
- ▶ Expor apenas as operações permitidas
- ▶ Interface $\times$ Implementação

Exemplo em C – Pilha:

```
1 typedef struct {  
2     int data[100];  
3     int top;  
4 } Stack;  
5  
6 void push(Stack *s, int value);  
7 int pop (Stack *s);
```

Benefícios: modularidade, manutenção, reutilização

Tipos Abstratos de Dados (TAD)

- ▶ Modelo matemático + conjunto de operações
- ▶ Independente da implementação
- ▶ Define o “contrato” da estrutura

Exemplos clássicos:

- ▶ Pilha (Stack)
- ▶ Fila (Queue)
- ▶ Lista (List)
- ▶ Árvore (Tree)

Quiz: Qual TAD é melhor para histórico de ações (undo)?

Tipos Abstratos de Dados (TAD)

- ▶ Modelo matemático + conjunto de operações
- ▶ Independente da implementação
- ▶ Define o “contrato” da estrutura

Exemplos clássicos:

- ▶ Pilha (Stack)
- ▶ Fila (Queue)
- ▶ Lista (List)
- ▶ Árvore (Tree)

Quiz: Qual TAD é melhor para histórico de ações (undo)?

Resposta: Pilha (LIFO)

Estrutura de Dados – Conceito Geral

- ▶ Estuda alternativas para gerenciar memória de forma eficiente
- ▶ Objetivos principais:
 - ▶ Menor uso possível de memória
 - ▶ Acesso mais rápido às informações
 - ▶ Melhor aproveitamento dos recursos
- ▶ Estruturas clássicas: vetores, listas, pilhas, filas, árvores, grafos...

Estrutura de dados eficiente = programa mais rápido e econômico

Ponteiros em C

- ▶ Ponteiro = variável que armazena **endereço de memória**
- ▶ Declaração: tipo *nome;
- ▶ Operadores fundamentais:
 - ▶ & → endereço de uma variável (*endereço de*)
 - ▶ * → conteúdo apontado pelo ponteiro (*valor em*)

Exemplo básico:

```
1  int a = 90;
2  int *mema = &a;          // mema guarda o endereço
                             de a
3  printf("%d\n", *mema);    // imprime 90
4  *mema = 100;              // altera a      a agora
                             vale 100
```

Exemplo – Troca com ponteiros (errado × certo)

Código errado (não troca):

```
1 void troca_errada(int i, int j) {  
2     int temp = i;  
3     i = j;  
4     j = temp;  
5 }
```

Exemplo – Troca com ponteiros (errado × certo)

Código errado (não troca):

```
1 void troca_errada(int i, int j) {  
2     int temp = i;  
3     i = j;  
4     j = temp;  
5 }
```

Código correto (passagem por referência):

```
1 void troca_certa(int *i, int *j) {  
2     int temp = *i;  
3     *i = *j;  
4     *j = temp;  
5 }
```

Exemplo – Troca com ponteiros (errado × certo)

Código errado (não troca):

```
1 void troca_errada(int i, int j) {  
2     int temp = i;  
3     i = j;  
4     j = temp;  
5 }
```

Código correto (passagem por referência):

```
1 void troca_certa(int *i, int *j) {  
2     int temp = *i;  
3     *i = *j;  
4     *j = temp;  
5 }
```

Uso: `troca_certa(&x, &y);`

Tipo Abstrato de Dados (TAD)

- ▶ Conjunto de **valores** + conjunto de **operações** sobre eles
- ▶ Separação clara: **o que fazer** $\times$ **como fazer**
- ▶ Vantagem: independência da implementação
- ▶ Modelo matemático (ignora limitações físicas iniciais)

Interface (contrato) vs Implementação

Exemplo de TAD em C – struct student

```
1 struct student {
2     char *name;
3     float marks;
4 };
5
6 int main() {
7     struct student student1;
8     student1.name = "Aline";
9     student1.marks = 5.6;
10
11     printf("Nome: %s\n", student1.name);
12     printf("Nota: %.1f\n", student1.marks);
13     return 0;
14 }
```

Exemplo de TAD em C – struct student

```
1 struct student {
2     char *name;
3     float marks;
4 };
5
6 int main() {
7     struct student student1;
8     student1.name = "Aline";
9     student1.marks = 5.6;
10
11     printf("Nome: %s\n", student1.name);
12     printf("Nota: %.1f\n", student1.marks);
13     return 0;
14 }
```

Exercício sugerido: crie TAD Aluno com cod, nome, nota1, nota2, nota3 e calcule média ponderada (0.2 + 0.3 + 0.5)

Listas Lineares

- ▶ Conjunto de elementos organizados sequencialmente (ordem lógica)
- ▶ Não necessariamente contíguos na memória
- ▶ Cada elemento = **nó** (ou nodo)
- ▶ Exemplos reais:
 - ▶ Fila de banco
 - ▶ Letras de uma palavra
 - ▶ Notas de alunos
 - ▶ Itens em estoque

Alocação de Memória e Formas de Agrupamento

Alocação estática

- ▶ Definida na compilação
- ▶ Vetores, structs simples

Alocação de Memória e Formas de Agrupamento

Alocação estática

- ▶ Definida na compilação
- ▶ Vetores, structs simples

Alocação dinâmica

- ▶ Durante execução (malloc, calloc)
- ▶ Listas encadeadas, árvores dinâmicas

Agrupamento

- ▶ **Sequencial:** células contíguas (vetor)
- ▶ **Encadeado:** nós com ponteiro para próximo

Alocação de Memória e Formas de Agrupamento

Alocação estática

- ▶ Definida na compilação
- ▶ Vetores, structs simples

Alocação dinâmica

- ▶ Durante execução (malloc, calloc)
- ▶ Listas encadeadas, árvores dinâmicas

Agrupamento

- ▶ **Sequencial:** células contíguas (vetor)
- ▶ **Encadeado:** nós com ponteiro para próximo

Nó = [dado | próximo]

Introdução às Estruturas de Dados

- ▶ Estruturas de dados organizam informação para otimizar operações
- ▶ Permitem inserção, remoção, busca e acesso eficiente
- ▶ Tipos comuns: Pilha, Fila, Lista, Árvores

Pergunta para reflexão: Qual a diferença entre estrutura estática e dinâmica?

Pilha (Stack) – Conceituação

- ▶ Estrutura LIFO (Last In, First Out)
- ▶ Operações principais:
 - ▶ `push()` – inserir elemento
 - ▶ `pop()` – remover elemento
 - ▶ `top()` / `peek()` – acessar elemento do topo
- ▶ Tipos:
 - ▶ Estática (vetor fixo)
 - ▶ Dinâmica (alocação variável com ponteiros)

Pilha – Exemplo em C (Estática)

```
1  #define SIZE 100
2  typedef struct {
3      int data[SIZE];
4      int top;
5  } Stack;
6
7  void push(Stack *s, int val){
8      if(s->top < SIZE-1) s->data[++s->top] = val;
9  }
10
11 int pop(Stack *s){
12     return s->top >= 0 ? s->data[s->top--] : -1;
13 }
```

Pergunta: O que acontece se fizermos push após a pilha estar cheia?

Fila (Queue) – Conceituação

- ▶ Estrutura FIFO (First In, First Out)
- ▶ Operações: enqueue(), dequeue(), front(), isEmpty()
- ▶ Tipos:
 - ▶ Estática (vetor fixo)
 - ▶ Dinâmica (alocação com ponteiros)

Fila – Exemplo em C (Circular Estática)

```
1  #define SIZE 5
2  typedef struct {
3      int data[SIZE];
4      int front, rear;
5  } Queue;
6
7  void enqueue(Queue *q, int val){
8      q->data[q->rear] = val;
9      q->rear = (q->rear+1)%SIZE;
10 }
11
12 int dequeue(Queue *q){
13     int val = q->data[q->front];
14     q->front = (q->front+1)%SIZE;
15     return val;
16 }
```

Pergunta: Qual a vantagem da fila circular sobre a fila estática simples?

Lista Simplesmente Encadeada

- ▶ Cada nó contém dado + ponteiro para próximo
- ▶ Operações: inserção, remoção, busca

```
1 typedef struct Node{  
2     int data;  
3     struct Node *next;  
4 } Node;
```

Pergunta: Como inserir um elemento no início da lista?

Lista Circular Simples

- ▶ Lista encadeada onde o último nó aponta para o primeiro
- ▶ Inserção, remoção e busca similar à lista simples

```
1 Node *head = NULL;  
2 // ltima ->next = head para fechar o c rculo
```

Pergunta: Qual a vantagem de usar lista circular para fila de atendimento?

Lista Duplamente Encadeada

- ▶ Cada nó contém ponteiros para o próximo e anterior
- ▶ Permite travessia em ambos os sentidos
- ▶ Operações: inserção, remoção, busca

```
1 typedef struct Node{  
2     int data;  
3     struct Node *prev, *next;  
4 } Node;
```

Pergunta: Como remover um nó sem percorrer toda a lista?

Lista Duplamente Circular

- ▶ Último nó aponta para o primeiro e o primeiro aponta para o último
- ▶ Aplicações: buffers circulares, jogos, playlists

1

```
// last->next = first; first->prev = last;
```

Pergunta: Qual a vantagem de usar lista duplamente circular?

Alocação de Espaço Variável

- ▶ Estruturas dinâmicas usam malloc/free
- ▶ Permitem crescimento e redução conforme necessidade

```
1 Node *n = (Node*) malloc(sizeof(Node));  
2 n->data = 10;  
3 free(n);
```

Pergunta: O que acontece se esquecer de liberar memória?

Árvore Genérica e Binária – Conceituação

- ▶ Estrutura hierárquica: cada nó tem filhos
- ▶ Árvores binárias: cada nó tem no máximo 2 filhos
- ▶ Aplicações: hierarquias, expressões, busca eficiente
- ▶ Operações: inserir, remover, percorrer (preorder, inorder, postorder)

Árvore Binária – Exemplo em C

```
1  typedef struct Node{
2      int data;
3      struct Node *left, *right;
4  } Node;
5
6  Node* insert(Node* root, int val){
7      if(!root){
8          Node* n = (Node*)malloc(sizeof(Node));
9          n->data = val; n->left = n->right = NULL
10             ;
11             return n;
12         }
13         if(val < root->data) root->left = insert(
14             root->left, val);
15         else root->right = insert(root->right, val);
16         return root;
17     }
```

Pergunta: Qual a sequência inorder de 20, 10, 30 inseridos nesta árvore?

Exercícios em Sala de Aula

- ▶ Pilha: Simular undo de operações e verificar topo da pilha
- ▶ Fila: Simular atendimento de clientes com fila circular
- ▶ Lista: Inserir, remover e buscar elementos em listas simples e duplas
- ▶ Árvore: Construir árvore binária e mostrar percursos preorder, inorder e postorder

Tempo sugerido: 40 minutos

Tipos Principais de Listas Lineares

- ▶ **Pilha (Stack)** → LIFO (Last In, First Out)
- ▶ **Fila (Queue)** → FIFO (First In, First Out)
- ▶ **Deque (Double-Ended Queue)** → entrada e saída em ambas extremidades

Próximos tópicos: detalhes de cada uma

Lista – Conceitos e Operações

Tipos principais:

- ▶ Sequencial (baseada em vetor / array)
- ▶ Encadeada (simples, dupla, circular)

Operações principais:

- ▶ Inserir (início, fim, posição)
- ▶ Remover (início, fim, valor)
- ▶ Buscar
- ▶ Tamanho / está vazia?

Exemplo – Nó de lista encadeada:

```
1 typedef struct Node {  
2     int          data;  
3     struct Node *next;  
4 } Node;
```

Lista – Exercício Resolvido

Problema: Inserir $10 \rightarrow 20 \rightarrow 30$ e depois remover 20

```
1 Node *head = NULL;  
2  
3 insert(&head, 10);  
4 insert(&head, 20);  
5 insert(&head, 30);  
6 remove(&head, 20);
```

Desenhe:

- ▶ Antes da remoção: $10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$
- ▶ Depois da remoção: $10 \rightarrow 30 \rightarrow \text{NULL}$

Pilha (Stack) – LIFO

Operações principais:

- ▶ `push()` – empilhar
- ▶ `pop()` – desempilhar
- ▶ `top()` – consultar topo
- ▶ `isEmpty()`

Aplicações típicas:

- ▶ Histórico de ações (undo)
- ▶ Verificação de parênteses
- ▶ Inversão de sequência
- ▶ Avaliação de expressões pós-fixa

Pilha – Exercício Resolvido

Verificar se parênteses estão balanceados: "(()())"

```
1 Stack s = {.top = -1};
2 char str[] = "(()())";
3
4 for(int i = 0; str[i]; i++) {
5     if (str[i] == '(') push(&s, 1);
6     else if (str[i] == ')') pop(&s);
7 }
8
9 printf("%s\n", s.top == -1 ? "Balanceada" : "Nao
    balanceada");
```

Quiz: O que acontece se a string começar com ')')'?

Pilha – Exercício Resolvido

Verificar se parênteses estão balanceados: "(()())"

```
1 Stack s = {.top = -1};  
2 char str[] = "(()())";  
3  
4 for(int i = 0; str[i]; i++) {  
5     if (str[i] == '(') push(&s, 1);  
6     else if (str[i] == ')') pop(&s);  
7 }  
8  
9 printf("%s\n", s.top == -1 ? "Balanceada" : "Nao  
    balanceada");
```

Quiz: O que acontece se a string começar com ')')'?

Tentativa de pop() em pilha vazia → erro!

Fila (Queue) – FIFO

(exemplo com fila circular)

```
1 #define SIZE 5
2 typedef struct {
3     int data[SIZE];
4     int front, rear;
5     int size;           // opcional: contador de
                        elementos
6 } Queue;
```

Operações principais:

- ▶ enqueue() – inserir no final
- ▶ dequeue() – remover do início
- ▶ front()
- ▶ isEmpty() / isFull()

Resumo Comparativo das Estruturas

Estrutura	Inserção	Remoção	Acesso	Busca
Lista encadeada	$O(1)^*$	$O(1)^*$	$O(n)$	$O(n)$
Lista sequencial	$O(n)$	$O(n)$	$O(1)$	$O(n)$
Pilha	$O(1)$	$O(1)$	$O(1)$	—
Fila (circular)	$O(1)$	$O(1)$	$O(1)$	—
Árvore binária BST	$O(\log n)$	$O(\log n)$	—	$O(\log n)$

* no início ou com ponteiro para fim

Atividade Prática (sugestão)

- ▶ **Lista:** Inserir 5 números, remover o menor, desenhar estados
- ▶ **Pilha:** Simular *undo* de 4 operações de texto
- ▶ **Fila:** Simular fila de impressão (3 documentos)
- ▶ **Árvore:** Inserir 5 números em BST e mostrar: preorder, inorder, postorder

Síntese da Aula

- ▶ Abstração → independência de implementação
- ▶ TAD → contrato claro de operações
- ▶ Lista encadeada → inserção/remoção eficiente no início
- ▶ Pilha → LIFO – undo, chamadas de função
- ▶ Fila → FIFO – ordem de chegada, buffers
- ▶ Árvore → hierarquia + busca mais eficiente (em média)

Próxima aula: Algoritmos sobre essas estruturas